pfi

Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

Course 3 Capstone

WEEK 3 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The scenario: a vendor's daily extract

The history to 30 June 2026 is clean: the Treasury par yield file and the synthetic spreads. The vendor's extract covers 1 July to 2 October, with a cover note.

The portfolio manager wants

The snapshot for 2026-10-02

The same snapshot for 2026-09-30

Built on data you have proved

Data problems are planted in the file. Nothing names them: the rules, the calendar, and your module find them.

```
To: Credit desk
From: Market data vendor, client operations
Subject: Daily extract, 2026-07-01 to 2026-10-02

The extract is attached as course3_capstone_extract.csv, in
the usual layout.

Period:        2026-07-01 to 2026-10-02
Market days:   66
Columns:       date, 2 Yr, 5 Yr, 10 Yr, 30 Yr (par yields,
percent), ig_spread_bp, hy_spread_bp (basis points)

Last values sent, for 2026-10-02:
  2 Yr     4.83 percent
  5 Yr     5.06 percent
  10 Yr    5.28 percent
  30 Yr    5.63 percent
  ig_spread_bp   97.0
  hy_spread_bp   328.7

The spread columns are synthetic course data, invented for
this course. They are not ICE, Moody's, or any index.
Please tell us if the extract does not agree with these
figures.
```

What you hand in: ten deliverables

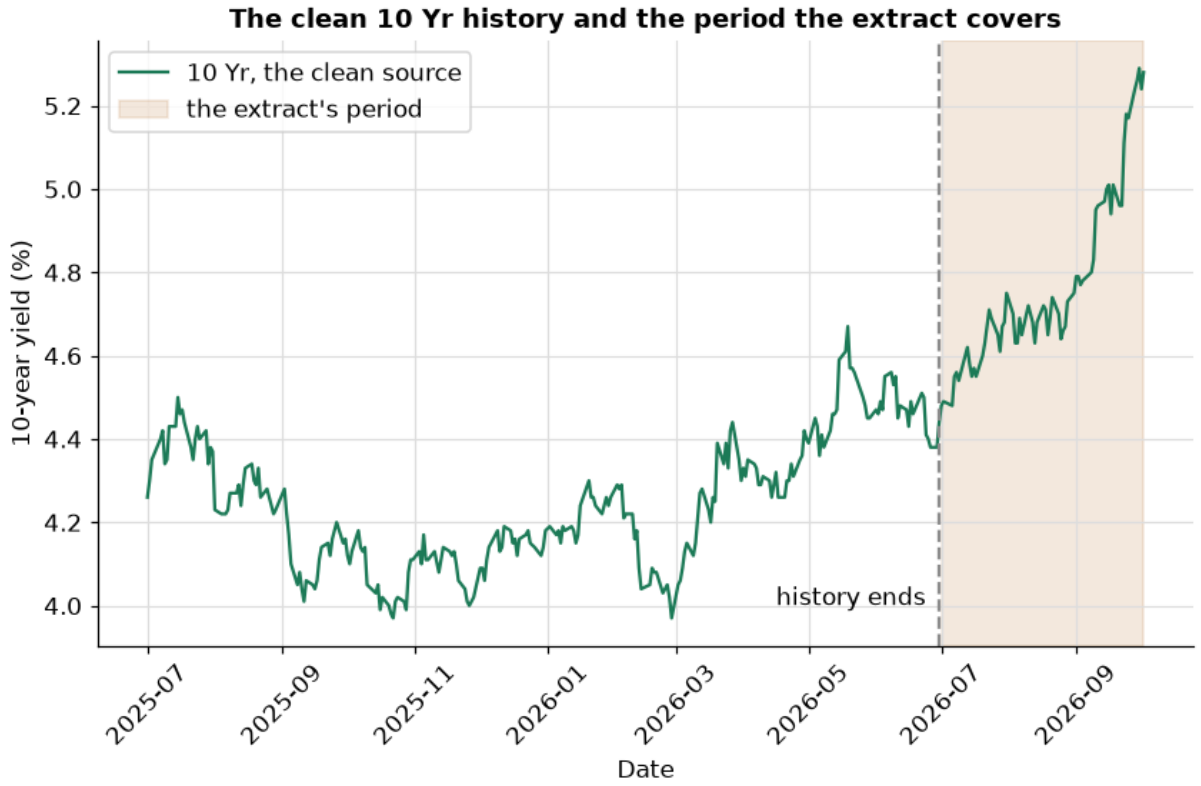
#	Deliverable	Section
1	marketdata.py complete, pytest -q passing, output saved	1
2	clean_extract(raw, source): decisions with asserts, and a log	2 to 5
3	Reconciliation: every difference a decision, 0 unexplained	6
4	The 2026-10-02 snapshot, HTML and Excel, read back	7
5	snapshot.py --as-of 2026-09-30, the dates side by side	8
6	Both levels tables in SQL, checked against pandas	8
7	One Git commit per file deliverable, then git tag course3	every section
8, 9	An AI-use log; a description that never forecasts	10
10	Optional, your own key: the snapshot from FRED	9

The files

File	What it is
`course3_capstone_extract.csv`	the vendor's extract: read every column as text
`course3_capstone_cover_note.txt`	the period, the market days, the last values
`treasury_par_yields.csv`	public domain: the history and the market calendar
`course3_synthetic_spreads.csv`	synthetic spreads, one row per Treasury date
`holdings.csv`	the September book, for the book section (day 14)

The course has the clean source for the extract's own period, so every decision can be proved by reconciliation. A real vendor's file has no clean twin.

The data you start from



The line is the clean source. Left of the dashed line the history is already clean. The shaded stretch is the vendor's extract, which you test, clean, and append.

6 more charts wait for you: each cell says "not attempted yet" until your answer exists, then draws it.

Ten sections, three sittings

First sitting

1 Your Complete Module

2 Test the Extract

3 Fix the Dates and the Rows

4 Fix the Values

Week 1: calendars, cleaning

Second sitting

5 Package: clean_extract

6 The Reconciliation

7 Append, Then the Snapshot

Weeks 2 and 3: measures

Third sitting

8 Rerun, Do Not Redo

9 Live (optional)

10 The Log and the Description

Week 3: SQL, AI, Git

Then the closing sections of every day: Excel Side by Side, Save the Day With Git, Bring It Home, and the AI check.

Testing the extract: the desk's eight rules

Rule	What the desk expects of an extract
1	One row for each market day of the period. No row is a copy of another
2	Every date is written YYYY-MM-DD
3	Every date is a market day: a date on which the Treasury file has a row
4	Every market day of the period has a row
5	No value is missing
6	Units are the cover note's: yields in percent, spreads in bp
7	No value stopped updating: no run of 5 or more identical values in a column
8	The cover note's figures: the number of market days, and the last values

Section 2 finds

Count on the extract as it arrived. Fix nothing yet.

Sections 3 and 4 fix

Dates first, then rows, then values. Evidence, then headline() before and after.

Matching the cover note is necessary, and it is not enough.

The market's own calendar

A market day is a row of the Treasury file

```
market_days = source.loc[first:last].index
# compare both ways
market_days.difference(dates)
dates.difference(market_days)
```

Not a weekday list, and not a holiday library (day 3).

Never fill from a neighbour

A forward fill writes a value the market never printed, and makes a change of exactly zero.

A value can be on a market day, in the right units, with nothing missing, and still be wrong. Day 3's stale_runs is there for a reason.

The vendor's rows are tested against the calendar, never the other way round.

How the checks work

A check holds a short code, never the answer

```
check('Question 2.1', answer_2_1, 'a short code')
```

```
correct
```

```
not yet. You have ...
```

```
not attempted yet
```

Decimals to the places asked. Dates as text YYYY-MM-DD. Several values in a tuple.

Closed blocks and two checkpoints

"Stuck after trying?" holds the decision the checkpoint expects for one kind of problem. Not a count, not an answer. Leave it closed until you have decided.

Checkpoints after section 4 and after clean_extract: every line correct before you go on.

40 questions, in the order an analyst works.

clean_extract, then the reconciliation

```
%%writefile clean_extract.py
def clean_extract(raw, source):
    """The decisions, in order ..."""
    # each decision of sections 3 and 4
    # one log line for every change
    # assert every rule, with a message
    return cleaned, log

cleaned, log = clean_extract.clean_extract(raw, source)
assert len(cleaned) == expected_days, "..."
```

Two comparisons

1 As received against the source files

2 Each difference: a decision from the log

3 Unexplained: the target is zero

4 Cleaned against the source files

Rules inside the function; the cover note asserted after the call.

Append, then the snapshot

Prove the append

```
history = source.loc[:HISTORY_END]
market = pd.concat([history, cleaned])
```

On the extract's columns, the market equals the source files, and every 252-row window across the join gives the same number.

snapshot.py, as on day 14

`build_market`

`daily_snapshot`

`snapshot_figures`

`write_snapshot`

`main  --as-of`

The extract carries four tenors; the history, the whole curve. Is each row true for the date it claims? (day 4)

Rerun, SQL, and Git

Rerun, do not redo

snapshot.py for 30 September, through subprocess: rebuilt from the files, nothing from memory. Then the two dates side by side.

SQL (days 11 and 12)

snapshot_history in course3.db; queries.sql with ? parameters; every query against pandas with assert_same_rows.

Git, all the way through

One commit per deliverable that writes a file. Snapshots and the database stay ignored. Then git tag course3.

```
Every command:  !git -C "{work}" ...
```

Section 9: live, with your own key (optional)

A live cell, two ways

No key (the saved notebook):

```
Live cell skipped: no FRED key was found.
```

With your key:

check lines: FRED equals the Treasury file

Five DGS tenors from FRED, checked against the Treasury file; the snapshot's levels from FRED's curve against those from the files.

ICE BofA and Moody's: metadata only

Rows, first and last date, dot rows, weekend rows. Values only if you set SHOW_LICENSED_VALUES = True on your own machine.

That is why the snapshot uses synthetic spreads: it is meant to be shared.

Nothing in section 9 is written to a file. Never print the key.

The log, the description, and the AI check

The AI-use log

What you asked, what you accepted or rejected, how you checked it. "none used" is valid. Never a key, firm data, or licensed data.

The description

300 to 450 words: levels, changes, one-year history, the book, the data. assert_descriptive is a reminder, not a proof.

The AI check

One step of clean_extract, in the style of an assistant's answer. It runs, with one data bug. Write the test first.

Describe, never forecast: what the data was, where it sits in its history, how it changed.

Code that runs is not the same as code that is right.

What you can do now

- Keep an API key out of code, notebooks, outputs, and Git
- Clean a series against the market's own calendar, never filled
- Changes, volatility, z-scores, ranks, slopes, flies: no lookahead
- SQL proved against pandas; an assistant's code tested first
- A snapshot that runs on any date, and describes, never forecasts

Next keep your my_bondmath folder up to date, and bring it with you.